

Towards a Methodology for Service Construction

Joachim Schelp

University of St. Gallen

Institute of Information Management

CH-9000 St. Gallen, Müller-Friedberg-Str. 8

Email: joachim.schelp@unisg.ch

Robert Winter

University of St. Gallen

Institute of Information Management

CH-9000 St. Gallen, Müller-Friedberg-Str. 8

Email: robert.winter@unisg.ch

Abstract— All large vendors of standard business software have announced to move to service oriented architecture. They claim benefits for their customers with regard to agility and openness. In contrast to the design of software services which has been a focus of research in the computer science community in recent years, the design of enterprise services has not been addressed as much by the information systems research community. This paper outlines a research program on the design of enterprise services. The necessity of different service understandings on different layers of enterprise architecture is shown, conceptual foundations for the design of enterprise services are presented, and first results of developing a method for enterprise service design are proposed.

I. INTRODUCTION

Over time, the information systems (IS) landscape of most companies evolves into a complex grid of applications. Its ever growing complexity aggravates efforts for maintaining alignment between business and IT structures when business requirements change or when technical innovations are implemented. Regardless whether adapted business artifacts (like processes, organizational structures) or adapted technical artifacts (like software systems, infrastructure components) trigger an update, the application landscape will have to be adapted in order to restore consistency. Service oriented architecture (SOA) aims at reducing the gravity of this problem: It is claimed that if functionalities are appropriately encapsulated as components which are loosely coupled, many changed business requirements can be met by re-composition of components rather than by alterations of implemented functionalities.

The discussion of service orientation is however often focused on implementation aspects (e.g. software design) and / or on specific implementation technologies (e.g. web services). It is however questionable whether a redesign of software components alone can effectively solve all change propagation problems in complex heterogeneous IS landscapes. A complexity problem this severe requires that not only software structures, but also business related structures are designed in a way that supports re-composition.

The number of contributions to service oriented architecture (SOA) is extensive, but also somehow limited [1]: While the number of papers is constantly growing, their main focus is on technical issues like protocols, messaging, service deployment within technical frameworks like J2EE, XML, or aspect oriented programming. A smaller fraction of papers

focuses on inter-organizational integration, typically in B2B e-business scenarios (e.g. [2], [3]). Only few papers go beyond the technical focus and interpret service orientation as a means to reorganize complex IS landscapes. The goals of service orientation from an enterprise-wide engineering perspective, and potential conflicts between SOA goals and other engineering goals (like re-use or alignment) have not been discussed explicitly so far.

Similar to the discussion of enterprise application integration (EAI), the integration perspective of services is often reduced to the interface complexity issue. To overcome the potential $n * (n - 1)$ two-way interconnections, services are proposed which are connected to each other using a service bus—1 : 1 coupling is replaced by $m : n$ coupling. Application cases show that loose coupling reduces interface complexity in complex IS architectures [4], [5]. But this effect does not necessarily reduce IS interdependencies. Complexity may even increase: Newly developed applications (often designated as services) are smaller in size in order to enable flexible coupling of such services into larger applications, thus increasing the number of applications and interfaces.

Due to business demands that are changing faster than the IS component grid can be altered consistently, variants of application components are introduced, and multiple variants are in use concurrently. If older component variants are referenced by newly developed services, the complexity of interdependencies often increases faster than in traditional software environments. Loose coupling addresses interface complexity, but still may create other problems.

The alignment problems created by different life cycles of business and system development are often addressed by introducing an additional abstraction layer. Companies like Deutsche Post [6] or Credit Suisse [5] as well as vendors like SAP with its Enterprise Service Architecture (ESA) are prominent examples for this (see [7], [8] as well). But there are indicators that an additional architectural layer alone is not sufficient to overcome all agility problems. A research program is therefore established that is aimed at proposing an integrated methodology to service construction based on the enterprise architecture (EA) framework that reflects the different life cycles on five different architectural layers [9]. This paper outlines the research proposal. The necessity of different service understandings on different EA layers is shown and the goals and goal conflicts in SOA are discussed. Furthermore

conceptual foundations for the design of enterprise services are described, and first components of a method for enterprise service design are presented.

II. CONCEPTUAL FOUNDATIONS

This section presents the underlying understanding of architecture II-A, service orientation II-B and its goals II-C which can be derived from the corporate goal system. The globalized, accelerating corporate environment demands for quick responses by any company—it requires agility [10], [11]. Agility demands however are not limited to strategy or organizational artifacts (e.g. processes, organizational structure) alone. The IS landscape, software systems, and infrastructure components have also to be agile. Agility as an IS / IT (re-)engineering goal can however only be achieved if software artifacts are regarded in the context of their usage to support business processes and ultimately to support business models. As a consequence, the scope of regarded architectures has to be widened from IS / IT artifacts to the entire enterprise architecture.

A. Enterprise Architecture

According to ANSI/IEEE Std 1471-2000, architecture is defined as the “fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution” [12]. EA therefore is understood as (1) the fundamental organization of a government agency or a corporation, either as a whole, or together with partners, suppliers and / or customers (“extended enterprise”), or in part (e.g. a division, a department, etc.) as well as (2) the principles governing its design and evolution. [13] While EA models represent as-is or to-be architectures of an actual corporation or government agency, respectively, an EA framework provides [13]

- one or more meta-model(s) for EA description,
- one or more method(s) for EA design and evolution,
- a common vocabulary for EA, and maybe even
- reference models that can be used as templates or blueprints for EA design and evolution.

The components of an EA framework should be applicable for a broad range of corporations and government agencies.

The above definition of architecture restricts included components to be “fundamental”. Due to the broad range of relevant component types, EA may nevertheless comprise a huge number of such artifacts. As a consequence, most EA frameworks distinguish several architecture layers and architecture views in order to reduce the number of artifacts per model [14], [15]. When several architecture layers and architecture views are differentiated, design and evolution principles have to address consistency and integration issues. The theory of hierarchical, multilevel systems [16] provides a conceptual foundation for such methods.

For EA, a hierarchical approach usually applies the “IT follows business” principle, starting with strategic positioning from the business management point of view, then deriving appropriate organizational processes and structures on this

basis, and finally specifying the information system, i.e. the interaction between human and technical information system components that appropriately support business requirements [17].

Most frameworks differentiate the following EA layers [18]:

Business architecture: The business architecture represents the fundamental organization of the corporation (or government agency) from a business strategy viewpoint. Typical artifacts represented on this layer are value networks, relationships to customer and supplier processes, targeted market segments, offered services, organizational goals, and strategic projects [19], [20]. Design and evolution principles for business architecture can be derived e.g. according to the market based approach [21] or the resource based approach [22] to strategic management.

Process architecture: The process architecture represents the fundamental organization of service development, service creation, and service distribution in the relevant enterprise context. Typical artifacts represented on this layer are business processes, organizational units, responsibilities, performance indicators, and informational flows [23]. Design and evolution principles for this layer focus on effectiveness (creating specified outputs) and efficiency (meeting specified performance goals) [24].

Integration architecture: The integration architecture represents the fundamental organization of information system components in the relevant enterprise context. Typical artifacts represented on this layer are enterprise services, application clusters, integration systems and data flows. Design and evolution principles for this layer focus on agility (e.g. by service orientation [25]), cost efficiency (e.g. by reduction of interfaces [26]), integration (e.g. by analysis of data coupling [27]), and / or speed (e.g. by straight-through processing [28]).

Software architecture: The software architecture represents the fundamental organization of software artifacts, e.g. software services and data structures. A broad range of design and evolution principles from computer science is available for this layer. **Technology (or infrastructure) architecture:** The technology architecture represents the fundamental organization of computing / telecommunications hardware and networks. A broad range of design and evolution principles from computer science is available for this layer too.

B. Service Orientation

Service oriented constructs are found on at least two different EA layers: If software components are implemented using web service standards like SOAP, UDDI and WSDL [29], they are designated as software services. If logical functionality representations are encapsulated into components which are designed in a loosely coupled fashion, they are designated as enterprise services. Enterprise services may be composed of more elementary enterprise services. Elementary enterprise services may be implemented by software services—as well as by traditionally implemented software components. Portions of business processes may be orchestrated from enterprise

services—as well as supported by traditionally designed applications.

C. Agility

According to the recent discussion in production management (e.g. [30]–[33]), agility goes beyond flexibility. While flexibility means that a system is able to be adapted to expected changes, agility means that a system is also adaptive to unexpected changes [34], [35]. Production management aims to “build in” flexibility by engineering for (re-)configurability of both production structures (e.g. by using CIM and CAD/CAM systems) and products (e.g. by component-based design).

(Re-)configurability however provides little help for unexpected changes. As a consequence, agility is recommended e.g. in [34], [36] not only related to production and product structures, but to the entire company. Yusuf et al. define this recommendation as follows: “Agility is the successful exploration of competitive bases (speed, flexibility, innovation proactivity, quality and profitability) through the integration of reconfigurable resources and best practices in a knowledge-rich environment to provide customer-driven products and services in a fast changing market environment” [32]. It is important to note that agility is not aimed at reacting to change, but as a proactive means to support change [30]. To stay competitive, agility has to be a goal for every company.

Corporate IT has an important impact on agility [11], [37]. IT supports faster product development cycles and enhanced flexibility of both production and product structures. In addition, IT can enable the development and implementation of new business models.

AHSAN AND YE-NGO [38] describe four maturity stages of the IT infrastructure itself and propose that the contribution to a company’s agility is the higher, the more advanced the IT infrastructure development stage is: (1) “Application silos” can be found in companies with large amounts of individual software systems that are tailored to certain business units. (2) The more elaborated “standardized technology” replaces such silos and supports several business units by shared IT solutions. (3) The IT infrastructure category “rationalized data” can be characterized as integrated and process oriented. (4) The most advanced IT infrastructure type “modular” requires IT solutions to be standardized, business oriented and marketable.

Infrastructure as described in [38]–[40] should be understood as a set of integrated applications and enterprise services rather than a specific technology. Many case studies support the assumption that service orientation contributes more to overall agility than any single technology [41]. However, the overall agility goal and its sub goal flexibility are conflicting with the re-use goal (e.g. [42]) within a service oriented architecture. This conflict is addressed in the next section.

III. RELATED WORK

Whereas agility is discussed broadly within the industrial engineering research community, there is no comparable discussion of agility in the context of IS and EA so far. Most IS and EA research is focusing on software architecture (e.g.

[15], [43]–[45]), e.g. is proposing construction guidelines and reference solutions for software artifacts, thereby pursuing software-related goals like reusability, component interchangeability or complexity reduction (e.g. [42], [46]).

A modular infrastructure, i.e. loosely coupled services which can be (re-)orchestrated in order to support changing business processes, requires that applications can be integrated and decoupled easily. This requirement is being discussed intensely in the literature—e.g. for enterprise application integration (EAI) in [47]–[51].

The recent discussion of SOA can be understood as a logical extension of the EAI discussion: While decoupling was discussed with reference to applications only in EAI, loose coupling and service orientation in terms of functionalities in general became the focus of the SOA discussion [52]. In contrast to the technology vendors’ focus on their latest (infrastructure) technologies and web service products, service orientation is now widely recognized as a design paradigm applying “equally well to the business as it does to software applications” [53]. As a consequence, the discussion of (re-)engineering for agility has to be extended from applications to all affected architectural levels.

Whereas design for agility on integration layer level is a new question and therefore covered sparse, contributions to flexibility on software and technology layer are numerous.

Component design on the integration layer aims to the construction of stable artifacts enabling a decoupling of solitary partial business processes on the process architecture level and corresponding software artifacts on the software architecture layer. But in the literature, the direct link between process and software layer is dominating the discussion (e.g. [54], [55]). If the integration layer is addressed [6], the discussion again focuses on technical aspects.

On the software layer itself there a rich discussion can be found on component design. It reflects not only technical issues like component communication (e.g. [56]–[58]), but also architectural issues (e.g. [47], [59]). The primary design principle visible in this discussion is reusability (e.g. [42], [46]). Research on e-business and B2B communication fostered an intense examination of web services. Again, architectural considerations are restricted to reusability, reducing interface complexity, and technical issues like protocols, languages, security aspects, and technical foundations (e.g. [60]–[71]). Gaps in web service technology compared to previous, more mature technologies like CORBA are addressed, and solutions to close these gaps are being developed. But agility issues beyond reducing interface complexity are not addressed. In particular, the above mentioned goal conflict between agility / flexibility and re-use becomes apparent here: Re-use of services increases their interdependencies. This in turn adds to the time required for applying changes if such changes alter interface signatures. In the short run, this can be avoided by providing alternative signatures, by signature mapping in the integration infrastructure, or by instantiating alternative versions of the services at run time depending on the severity of the alteration to be implemented. But in the long run, the

number of alternative versions has to be restricted—because otherwise no sufficient re-use can be achieved. When this point is reached, the time buffer is consumed and any additional alteration requires changes within the dependent services as well.

Summing up, existing literature does not sufficiently address the integration layer and its importance for decoupling business related structures on the one hand, and IT related structures on the other. This decoupling however is a necessary precondition for buffering changes and supporting alignment, hence for contributing to agility on a sustainable level [72]. Enterprise design frameworks like ARIS and the Business Engineering framework [73] are currently reworked to address the issues coming up with the discussion of service orientation. This paper contributes to the Business Engineering framework. The intended construction methodology is effective on the integration layer and supports the design of appropriate enterprise services.

IV. METHOD CONSTRUCTION

The relevance of the intended methodology is obvious: SOA technologies do not address the ever growing complexity of IS landscapes. Compared to changing business requirements, adaptations of application architecture are usually lagging behind. Existing application design methodologies resulted in the high complexity of today's IS landscapes so that they are not generally suitable for the design of enterprise services.

As a consequence, a methodology for enterprise service construction cannot be solely derived from literature or case studies—only few companies have already experience and know-how in enterprise service design, and even less are successful. Since empirical approaches are not applicable, a design research approach is chosen. A suitable research process has e.g. proposed by ROSSI AND SEIN [74]: (1) Identify a need. (2) Build a methodology with respect to good design research principles, usage of available good practices and the definition of the measures of success. (3) Evaluate the methodology. (4) Learn and Theorize.

(ad 1) The first step was sketched above: Existing application development approaches fail when increasing agility is considered as a goal. In practice increasing complexity can be observed when SOA is introduced. Since previous research does not address the problem at hand, the need for an alternative approach in service construction is rationalized.

(ad 2) The design of the methodology has to reflect existing practice, design research principles [75], and the definition of measures of success. Existing practice is gathered in several steps: Focus groups with experienced enterprise architects of major companies are set up to unveil current practice in service design. As services can be designed top-down (deriving from processes) and bottom-up (composing existing services to new services), both practices have to be analyzed. Expert interviews as well as surveys are conducted in this step.

(ad 3) The methodology is to be developed according to method engineering. The resulting method is comprised of reference procedure models, reference model instances ("lead

elements"), a documentation model, a role model, and a meta model. Documentation and role models give additional advice how to apply the methodology regarding the documents types to be produced and the resources required to apply the method. The meta model specifies the conceptual data model of the results, thereby guaranteeing the consistency of the entire method.

For the reference procedure model, research is currently conducted to gather service design (top-down) and service composition (bottom-up) practices and to develop new approaches.

TABLE I
GUIDELINES FOR SERVICE COMPOSITION

No.	Directive	Description	Category
1	Process dominance	Service design follows business process design	Reference
2	Process scope	Any orchestrated service has to cover a distinct (partial) business process	
3	Intersection points	Service intersection points refer to business process intersection points	
4	Interface reference	Interfaces are derived from a higher level business object model	
5	Object scope	Any atomic service refers to one business object only	Structure
6	Service type	Tightly coupled services are composed, loosely coupled services are orchestrated	
7	Completeness	Any service is complete regarding processes and business objects involved	
8	Service coverage	Any service encapsulates stable and atomic business object and function blocks	
9	Service depth	Composed services must not export methods of underlying services 1:1	
10	Re-use	Only the newest variant of any existing service can be referenced by newly (re-) designed services. Older variants have to be decommissioned.	
11	Ownership	Every service has one distinct owner, even if underlying services have different owners.	Ownership
12	Data ownership precedence	Data ownership has priority in the allocation of new ownerships when considering underlying services	

For service composition, first results are available: In order to transform existing integration architectures into SOA, appropriate service composition guidelines have to be observed. In order to identify such guidelines, experienced architects from eight large companies (six financial service providers, a telecommunications provider, and a power utility provider)

have been invited to a workshop. All companies have experience in SOA design and operations. In some cases the service oriented architecture implemented is huge in size (> 100 resp. > 700 services), and in these cases alternative versions of the same service being run do occur.

An initial focus group of four senior architects was asked to define design guidelines for service composition. The brainstorming process was moderated. The results were discussed in an open discussion round by all participating architects. A set of those twelve directives was derived which were agreed upon by all participants. These directives are exhibited in I.

The guidelines for service composition can be clustered into a *reference*, a *structure*, and an *ownership* group. Services have to be composed with *reference* to processes (on the process architecture layer) and to functions and data objects (on the integration architecture layer). Process dominance defines that the design of all services has to reflect business processes and their requirements. Artifacts designed only according to technical requirements are not considered to be (enterprise) services—these artifacts are designated as software services (or *technical* services) to stress the conceptual difference.

The *structure* of the service itself has to reflect the following guidelines: An atomic service deals with one business object only—with business objects reflecting the enterprise data model from a business perspective. Non-atomic services can be orchestrated or composed from other services. Orchestration is defined as the combination of services at run-time, while composition of services is defined as their combination at build time. Due to IT infrastructure complexity, both types of service combinations are found in most companies. As a consequence, loosely coupled services have to be orchestrated, tightly coupled services have to be composed (service type). Considering that each service supports a (partial) business process, it has to be complete regarding the (computer supportable) activities in the process and the business objects involved. The data structures and function blocks affected have to be atomic and stable (service coverage). Otherwise they have to reference other services. Conversely a non-atomic service must not export an underlying service one by one (service depth). It has to add functionality or data compared to the underlying services. If variants of a service are built, only the newest version of this service can be referenced when further new services are designed. Accordingly during the maintenance of a service, it has to reference the newest version of every underlying / used service. Only when this guideline is observed, an uncontrolled growth of service variants can be confined. Otherwise the (long-term) goal of increasing service re-use cannot be reached. Implementing any business demand too fast is restricted by this, as is the short term goal of agility.

Finally every service has to have a distinct owner (*ownership*). If a service is combined of other services, the data ownerships of these services have to be considered, and the ownership of the new service defined accordingly (data ownership precedence).

For top-down service design a similar research setup is scheduled. The results will be compared and tested against

a broader survey.

To complement the proposed method, the procedure model has to be complemented by activity and role specifications and accompanying governance structures, which are the more important when contributions to agility are to be addressed [76]. A series of interviews with experts is scheduled for this purpose. Furthermore, an information model is being developed with regard to the relevant EA components, especially on the integration and the software layers.

In the further course of the method development process, success measures need to be defined in order to evaluate the laboratory or field implementations.

V. OUTLOOK

The outlined enterprise service construction method constitutes the core of a research program which runs from February 2006 to March 2007. Its aim is to contribute to the Business Engineering framework [73] in order to better reflect service orientation on all architectural layers consistently, and in particular to address goal conflicts regarding agility and re-use. In accordance with design research principles, interviews and literature studies are used to derive a generic artifact, while instantiation—either in the laboratory or in the field—will be needed to evaluate the method. Initial results show promise. First interviewees in the preparation work of the survey agree on the set of directives gathered. Furthermore the architectural views and layers presented find acceptance.

During summer 2006 work on directives will be concluded, the other steps mentioned above will be finalized before March 2007. After this the *learn and theorize* phase within the research cycle can start to generalize findings and to confirm or reject the original assumptions. Until then current and upcoming findings can give academics and practitioners interested in the topic an overview of the current state in service design. Furthermore it is an application of the research process proposed by ROSSI AND SEIN [74], resulting finally not only in better theories on service design, but providing feedback to ongoing discussions in method engineering and research processes in design science.

REFERENCES

- [1] G. Brown and R. Carpenter, "Successful Application of Service-Oriented Architecture Across the Enterprise and Beyond," *Intel Technology Journal*, vol. 8, no. 4, pp. 345–359, Nov. 2004.
- [2] J. M. Tenenbaum and R. Khare, "Business Services Networks: delivering the promises of B2B," in *Proceedings of the IEEE IEEE05 International workshop on Business services networks (BSN'05)*, March 29, 2005, Hong Kong. Piscataway, NJ, USA: IEEE Press, 2005, pp. 8–8.
- [3] T. W. Tewoldeberhan, A. Verbraeck, and S. S. Msanjila, "Simulating process orchestrations in business networks: a case using BPEL4WS," in *Proceedings of the 7th international conference on Electronic commerce (ICEC'05)*, August 15–17, 2005, Xi'an, China, ser. ACM International Conference Proceeding Series, Q. Li and T.-P. Liang, Eds., vol. 113. New York, NY, USA: ACM Press, 2005, pp. 471–477.
- [4] T. Puschmann, R. Alt, and D. Sassmannshausen, "Enterprise Application Integration bei Robert Bosch," in *Business Networking in der Praxis – Beispiele und Strategien zur Vernetzung mit Kunden und Lieferanten*, ser. Business Engineering, H. Osterle, E. Fleisch, and R. Alt, Eds. Berlin et al.: Springer, 2002, ch. 14, pp. 271–298.

- [5] C. Hagen, "Integrationsarchitektur der Credit Suisse," in *Enterprise Application Integration – Flexibilisierung komplexer Unternehmensarchitekturen*, S. Aier and M. Schönherr, Eds. Berlin: GITO-Verlag, 2003, pp. 61–81.
- [6] M. Herr, U. Bath, and A. Koschel, "Implementation of a Service Oriented Architecture at Deutsche Post MAIL," in *Proceedings of the European Conference on Web Services (ECOWS 2004), September 27-30 2004, Erfurt, Germany*, ser. Lecture Notes in Computer Science (LNCS), L.-J. Zhang and M. Jeckle, Eds., vol. 3250. Berlin et al.: Springer, 3-540-23202-8 2004, pp. 227–238.
- [7] Z. Duan, S. Bose, P. A. Stirpe, C. Shoniregun, and A. Logvynovskiy, "SOA without Web services: a pragmatic implementation of SOA for financial transactions systems," in *Proceedings of the 2005 IEEE International Conference on Services Computing (SCC2005), July 11-15 2005, Orlando, Florida*, vol. 1. Los Alamitos, CA, USA: IEEE Computer Society Press, July 2005, pp. 243–250.
- [8] R. Heutschi, C. Legner, and H. Österle, "Serviceorientierte Architekturen – Vom Konzept zum Einsatz in der Praxis," in *Integration, Informationslogistik und Architektur – Proceedings der DW2006, September 21-22, Friedrichshafen, Germany*, ser. Lecture Notes in Informatics, J. Schelp, R. Winter, U. Frank, B. Rieger, and K. Turowski, Eds., vol. P90. Bonn: Gesellschaft für Informatik GI, Sept. 2006, pp. 361–382.
- [9] A. Dietzsch, "Architekturmanagement – Rahmen und Realisierung der Unternehmensarchitektur der Mobiliar," in *Integrationsmanagement – Planung, Bewertung und Steuerung von Applikationslandschaften*, ser. Business Engineering, J. Schelp and R. Winter, Eds. Berlin et al.: Springer, 2006, pp. 231–266.
- [10] R. Gandossy, "The Need for Speed," *Journal of Business Strategy*, vol. 24, no. 1, pp. 29–33, 2003.
- [11] Q. Cao and S. Dowlatshahi, "The impact of alignment between virtual enterprise and information technology on business performance in an agile manufacturing environment," *Journal of Operations Management*, vol. 23, no. 55, pp. 531–550, 2005.
- [12] IEEE, *IEEE Recommended Practice for Architectural Description of Software Intensive Systems*, The Institute of Electrical and Electronics Engineers, Inc. IEEE Std 1471-2000, 2000. [Online]. Available: <http://standards.ieee.org/reading/ieee/std/se/1471-2000.pdf>
- [13] Opengroup, "TOGAF "Enterprise Edition" Version 8.1," The Open Group, 2003. [Online]. Available: <http://www.opengroup.org/architecture/togaf8-doc/arch/>
- [14] J. Scheckerman, *How to Survive in the Jungle of Enterprise Architecture Frameworks: Creating or Choosing an Enterprise Architecture Framework*, 2nd ed. Victoria, British Columbia: Trafford Publishing, 2004. [Online]. Available: http://www.enterprise-architecture.info/EA_Book_EAFrameworks.htm
- [15] A. Tang, J. Han, and P. Chen, "A Comparative Analysis of Architecture Frameworks," Swinbourne University of Technology, Centre for Component Software and Enterprise Systems, Swinbourne, Technical Report SUTIT-TR2004.01, 2004 2004. [Online]. Available: <http://www.it.swin.edu.au/centres/TechnicalReports/2004/SUTIT-TR2004.01.pdf>
- [16] M. D. Mesarovic, D. Macko, and Y. Takahara, *Theory of Hierarchical, Multilevel Systems*. New York, London: Academic Press, 1970.
- [17] C. Braun and R. Winter, "A Comprehensive Enterprise Architecture Metamodel and Its Implementation Using a Metamodeling Platform," in *Enterprise Modelling and Information Systems Architectures – Proceedings of the Workshop in Klagenfurt, October 24-25, 2005*, ser. Lecture Notes in Informatics (LNI) – Proceedings, Series of the Gesellschaft für Informatik (GI), J. Desel and U. Frank, Eds., vol. P-75. Bonn: Gesellschaft für Informatik, 2005, pp. 64–79.
- [18] R. Winter and R. Fischer, "Essential Layers, Artifacts, and Dependencies of Enterprise Architecture," in *EDOC Workshop on Trends in Enterprise Architecture Research (TEAR 2006), October 17 2006, Hong Kong*, I. C. Society, Ed. Los Alamitos, CA, USA: IEEE Computer Society Press, 2006.
- [19] J. Hedman and T. Kalling, "The business model concept: theoretical underpinnings and empirical illustrations," *European Journal of Information Systems*, vol. 12, no. 1, pp. 49–59, Mar. 2003.
- [20] P. Weill and M. R. Vitale, *Place to Space: Migrating to eBusiness Models*. Boston, Massachusetts: Harvard Business School Press, 2001.
- [21] M. E. Porter, *Competitive Advantage: creating and sustaining superior performance*, 2nd ed. New York: Free Press, 1998.
- [22] G. Hamel and C. K. Prahalad, "The Core Competence of the Corporation," *Harvard Business Review*, vol. 68, no. 3, pp. 79–91, 1990.
- [23] T. H. Davenport, *Process Innovation – Reengineering Work through Information Technology*. Boston: Harvard Business School Press, 1993.
- [24] H. Österle, *Business in the Information Age – Heading for New Processes*. New York: Springer, 1995.
- [25] K. B. Douglas, *Web Services and Service-Oriented Architecture: Your Road Map to Emerging IT*. Morgan Kaufmann Publishers, 2003.
- [26] D. S. Linthicum, *Enterprise Application Integration*, ser. Addison-Wesley Information Technology Series. Harlow et al.: Addison-Wesley, 2000.
- [27] IBM, "Business Systems Planning – Information Systems Planning Guide," IBM, Atlanta, Working Report IBM-Form GE20-0527-4, 1984.
- [28] B. Kuhlman and H. Thielmann, Eds., *The Practical Real-Time Enterprise: Facts and Perspectives*. Springer Verlag, 2005.
- [29] A. Tsalgaidou and T. Pilioura, "An Overview of Standards and Related Technology in Web Services," *Distributed and Parallel Databases*, vol. 12, no. 2-3, pp. 135–162, Sep 2002, 0926-8782.
- [30] S. L. Goldman, R. N. Nagel, and K. Preiss, *Agile competitors and virtual organizations: strategies for enriching the customer*. New York: Van Nostrand Reinhold, 1995.
- [31] H. Sharifi and Z. Zhang, "A methodology for achieving agility in manufacturing organisations: An introduction," *International Journal of Production Economics*, vol. 62, no. 1-2, pp. 7–22, 1999.
- [32] Y. Y. Yusuf, M. Sarhadi, and A. Gunasekaran, "Agile manufacturing: the drivers, concepts and attributes," *International Journal of Production Economics*, vol. 62, no. 1-2, pp. 33–43, 1999.
- [33] Z. Zhang and H. Sharifi, "A methodology for achieving agility in manufacturing organisations," *International Journal of Operations & Production Management*, vol. 20, no. 4, pp. 496–512, 2000.
- [34] F. Becker, "Organisational agility and the knowledge infrastructure," *Journal of Corporate Real Estate*, vol. 3, no. 1, pp. 28–37, 2001.
- [35] R. J. Vokurka and G. Fliedner, "The journey toward agility," *Industrial Management & Data Systems*, vol. 98, no. 4, pp. 165–171, 1998.
- [36] C. R. Duguay, S. Landry, and F. Pasin, "From mass production to flexible/agile production," *International Journal of Operations & Production Management*, vol. 17, no. 12, pp. 1183–1195, 1997.
- [37] V. Sambamurthy, A. Bharadwaj, and V. Grover, "Shaping Agility through Digital Options: Reconceptualizing the Role of Information Technology in Contemporary Firms," *MIS Quarterly*, vol. 27, no. 2, pp. 237–263, 2003.
- [38] M. Ahsan and L. Ye-Ngo, "The Relationship between I.T. Infrastructure and Strategic Agility in Organizations," in *Proceedings of the Eleventh Americas Conference in Information Systems (AMCIS2005), August 11-14, 2005, Omaha, NE, USA*, J. Romano, Nicholas C., Ed. Atlanta, Georgia, USA: Association for Information Systems, 2005, pp. 415–427.
- [39] P. Weill, M. Subramani, and M. Broadbent, "IT Infrastructure for Strategic Agility," Massachusetts Institute of Technology (MIT) – Sloan School of Management, Cambridge, MA, Working Paper 4235-02, Apr. 2002. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.317307>
- [40] A. Umar, "IT Infrastructure to Enable Next Generation Enterprises," *Information Systems Frontiers*, vol. 7, no. 3, pp. 217–256, 2005.
- [41] A. E. Coronado Mondragon, A. C. Lyons, and D. F. Kehoe, "Assessing the value of information systems in supporting agility in high-tech manufacturing enterprises," *International Journal of Operations & Production Management*, vol. 24, no. 12, pp. 1219–1246, 2004.
- [42] R. Berberner, T. Grollius, N. Repp, O. Heckmann, E. Ortner, and R. Steinmetz, "An approach for the Management of Service-oriented Architecture (SoA) based Application Systems," in *Enterprise Modelling and Information Systems Architectures – Proceedings of the Workshop in Klagenfurt, October 24-25, 2005*, ser. Lecture Notes in Informatics (LNI) – Proceedings, Series of the Gesellschaft für Informatik (GI), J. Desel and U. Frank, Eds., vol. P-75. Bonn: Gesellschaft für Informatik, 2005, pp. 208–221.
- [43] W. Hasselbring, "Information system integration," *Communications of the ACM*, vol. 43, no. 6, pp. 32–38, 2000.
- [44] T. Zimmermann, S. Diehl, and A. Zeller, "How history justifies system architecture (or not)," in *Proceedings of the Sixth International Workshop on Principles of Software Evolution, Sep 1-2, 2003, Helsinki*, T. Mikko-nen, M. W. Godfrey, and M. Saeki, Eds. Los Alamitos, CA: IEEE Computer Society Press, 2003, pp. 73–83.
- [45] N. Medvidovic, D. S. Rosenblum, and R. N. Taylor, "A language and environment for architecture-based software development and evolution," in *Proceedings of the 21st international conference on Software engineering (ICSE'99), May 16-22, 1999, Los Angeles, CA*, B. Boehm,

- D. Garlan, and J. Kramer, Eds. Los Alamitos, CA, USA: IEEE Computer Society Press, 1999, pp. 44–53.
- [46] E. M. Dashofy, N. Medvidovic, and R. N. Taylor, “Using off-the-shelf middleware to implement connectors in distributed software architectures,” in *Proceedings of the 21st international conference on Software engineering (ICSE '99)*. Los Alamitos, CA, USA: IEEE Computer Society Press, 1999, pp. 3–12.
- [47] A. Arsanjani, “Developing and integrating enterprise components and services,” *Communications of the ACM*, vol. 45, no. 10, pp. 31–34, 2002.
- [48] Z. Irani, M. Themistocleous, and P. E. D. Love, “The impact of enterprise application integration on information system lifecycles,” *Information Management*, vol. 41, no. 2, pp. 177–187, 2003. [Online]. Available: [http://dx.doi.org/10.1016/S0378-7206\(03\)00046-6](http://dx.doi.org/10.1016/S0378-7206(03)00046-6)
- [49] J. Sutherland and W.-J. van den Heuvel, “Enterprise Application Integration and Complex Adaptive Systems,” *Communications of the ACM*, vol. 45, no. 10, pp. 59–64, 2002. [Online]. Available: <http://doi.acm.org/10.1145/570907.570932>
- [50] J. Schelp and A. Schwinn, “Extending the Business Engineering Framework for Application Integration Purposes,” in *Proceedings of the 2005 ACM symposium on Applied computing (SAC2005)*, March 13–17, 2005, Santa Fe, New Mexico, USA, ACM, Ed. Santa Fe, New Mexico, USA: ACM Press, 13.03.2005 2005, pp. 1333–1337.
- [51] A. Schwinn and R. Winter, “Success Factors and Performance Indicators for Enterprise Application Integration,” in *Proceedings of the Eleventh Americas Conference on Information Systems (AMCIS 2005)*, August 11th–14th 2005, J. Romano, Nicholas C., Ed. Omaha, NE: Association for Information Systems, 11.08.2005 2005, pp. 2179–2189.
- [52] M. Stal, “Using Architectural Patterns and Blueprints for Service-Oriented Architecture,” *IEEE Software*, vol. 23, no. 2, pp. 54–61, 2006.
- [53] M. Lankhorst, *Enterprise Architecture at Work: Modelling, Communication and Analysis*. Berlin Heidelberg: Springer Verlag, 2005.
- [54] A. Albani and J. M. Zaha, “From Reference Model to Component Model,” in *Enterprise Modelling and Information Systems Architectures – Proceedings of the Workshop in Klagenfurt, October 24–25, 2005*, ser. Lecture Notes in Informatics (LNI) – Proceedings, Series of the Gesellschaft für Informatik (GI), J. Desel and U. Frank, Eds., vol. P-75. Bonn: Gesellschaft für Informatik, 2005, pp. 22–35.
- [55] K. Turowski, “Spezifikation und Standardisierung von Fachkomponenten,” *Wirtschaftsinformatik*, vol. 43, no. 3, pp. 269–281, 2001.
- [56] H. Gomaa and D. A. Menascé, “Design and performance modeling of component interconnection patterns for distributed software architectures,” in *Proceedings of the second international workshop on Software and performance (WOSP'00)*, September 17–20 2000, Ottawa, Ontario, Canada, M. Woodside, H. Gomaa, and D. Menasce, Eds. New York, NY, USA: ACM Press, 2000, pp. 117–126.
- [57] H. Gomaa, D. A. Menascé, and M. E. Shin, “Reusable component interconnection patterns for distributed software architectures,” in *Proceedings of the 2001 Symposium on Software reusability: putting software reuse in context (SSR'01)*, May 18–20, 2001, Toronto, Ontario, Canada, P. G. Basset, Ed. New York, NY, USA: ACM Press, 2001, pp. 69–77, this article has also been published in *ACM SIGSOFT Software Engineering Notes*, Volume 26, Issue 3 (May 2001), p. 69–77, ISSN 0163-5948.
- [58] Y. Jin and J. Han, “PEIDL: An Interaction Protocol Specification Language for Software Components,” Swinbourne University of Technology, Centre for Component Software and Enterprise Systems, Swinbourne, Technical Report SUTIT-TR2004.02/SUT.CeCSes-TR002, Oct. 2004. [Online]. Available: <http://www.it.swin.edu.au/centres/TechnicalReports/2004/SUTIT-TR2004.02.pdf>
- [59] V. R. Basili, G. Caldiera, and G. Cantone, “A reference architecture for the component factory,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 1, no. 1, pp. 53–80, 1992.
- [60] A. White, E. M. Daniel, and M. Mohdzain, “The role of emergent information technologies and systems in enabling supply chain agility,” *International Journal of Information Management*, vol. 25, no. 5, pp. 396–410, 2005. [Online]. Available: <http://dx.doi.org/10.1016/j.ijinfomgt.2005.06.009>
- [61] R. Yuan, L. Zunchao, F. Boqin, and H. Jincang, “Architecture-based Web service composition framework and strategy,” in *Proceedings of the 12th IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS'05)*, April 4–5 2005, Greenbelt, Maryland, USA, M. Hinchey, J. Leaney, J. Rozenblit, and T. O'Neill, Eds. IEEE Computer Society Press, Apr. 2005, pp. 129–134.
- [62] W. M. P. van der Aalst, M. Dumas, and A. H. M. ter Hofstede, “Web service composition languages: old wine in New bottles?” in *Proceedings of the 29th Euromicro Conference, 2003, Sep 1–6 2003, Belek-Antalya, Turkey*, G. Chroust and C. Hofer, Eds. Los Alamitos, CA, USA: IEEE Computer Society Press, September 2003, pp. 298–305.
- [63] I. Altintas, E. Jaeger, K. Lin, B. Ludaescher, and A. Memon, “A Web service composition and deployment framework for scientific workflows,” in *Proceedings of the IEEE International Conference on Web Services (ICWS'04)*, July 6–9 2004, San Diego, CA, USA, L.-J. Zhang, Ed. Los Alamitos, CA, USA: IEEE Computer Society Press, 6–9 July 2004, pp. 814–815.
- [64] B. Benatallah, Q. Z. Sheng, and M. Dumas, “The Self-Serv environment for Web services composition,” *IEEE Internet Computing*, vol. 7, no. 1, pp. 40–48, Jan.-Feb. 2003.
- [65] D. Benslimane, Z. Maamar, and C. Ghedira, “A view-based approach for tracking composite Web services,” in *Proceedings of the Third IEEE European Conference on Web Services (ECOWS2005)*, Nov 14–16 2005, Växjö, Sweden, W. Löwe and J.-P. Martin-Flatin, Eds. Los Alamitos, CA, USA: IEEE Computer Society Press, Nov. 2005, pp. 1–10.
- [66] B. Carminati, E. Ferrari, and P. C. K. Hung, “Web Service Composition: A Security Perspective,” in *Proceedings of the International Workshop on Challenges in Web Information Retrieval and Integration (WIRI'05)*, April 8–9 2005, Tokyo, Japan, J. Adachi, W. Shan, and A. Vakali, Eds. Los Alamitos, CA, USA: IEEE Computer Society Press, 08–09 April 2005, pp. 248–253.
- [67] M. De Backer, G. Dedene, and J. Vandelbulcke, “Web services composition, execution and visualization,” in *Proceedings of the 12th IEEE International Workshop on Program Comprehension (IWPC'04)*, June 24–26 2004, Bari, Italy, G. Visagio, A. K. Amschler Andrews, and F. Lanubile, Eds. Los Alamitos, CA, USA: IEEE Computer Society Press, 24–26 June 2004, pp. 264–265.
- [68] J. A. Fisteus, C. D. Kloos, and L. Sanchez Fernández, “Verification of Web Services Compositions: Applying Model Checking to BPEL4WS,” *Latin America Transactions, IEEE (Revista IEEE America Latina)*, vol. 3, no. 1, pp. 1–8, Mar. 2005. [Online]. Available: http://ieeexplore.ieee.org/xpl/freeabs_all.jsp?isnumber=31504&arnumber=1468660&count=19&index=5
- [69] R. Hull, “Web Services Composition: A Story of Models, Automata, and Logics,” in *Proceedings of the 2005 IEEE International Conference on Services Computing (SCC2005)*, July 11–15 2005, Orlando, Florida, C. K. Chang, L.-J. Zhang, S. Purao, H. Jin, and F. Leymann, Eds., vol. 1. Los Alamitos, CA, USA: IEEE Computer Society Press, July 2005, pp. xviii–xix.
- [70] M. zur Muehlen, J. V. Nickerson, and M. Weske, “Web services and workflow: composition, collaboration, coordination,” in *Proceedings of the 37th Annual Hawaii International Conference on Systems Sciences (HICSS'04)*, Jan 5–8 2004, Hawaii, USA, R. H. Sprague, Ed. Los Alamitos, CA, USA: IEEE Computer Society Press, Jan. 2004, p. 1.
- [71] D. Skogan, R. Groenmo, and I. Solheim, “Web service composition in UML,” in *Proceedings of the Eighth IEEE International Enterprise Distributed Object Computing Conference (EDOC 2004)*, September 20–24 2004, Monterey, CA, USA, D. H. Akehurst, M. van Sinderen, and B. R. Bryant, Eds. Los Alamitos, CA, USA: IEEE Computer Society Press, 2004, pp. 47–57.
- [72] S. Aier, “EAI und Nachhaltigkeit von Architekturen – Ergebnisse einer empirischen Studie,” in *Auf dem Weg zur Integration Factory – Proceedings der DW2004 – Data Warehousing und EAI*, Nov 3–4 2004, Friedrichshafen, Germany. Heidelberg: Physica, 04.11.2004 2004, pp. 41–58.
- [73] H. Österle and R. Winter, Eds., *Business Engineering*, 2nd ed., ser. Business Engineering. Berlin et al.: Springer, 2003.
- [74] M. Rossi and M. K. Sein, “Design Research Workshop: A Proactive Research Approach,” 2003, presentation given at the Design Research Workshop within the IRIS26. [Online]. Available: <http://tiesrv.hkkk.fi/iris26/presentation/workshop.designRes.pdf>
- [75] B. Niehaves, M. Ribbert, A. Dreiling, and R. Holten, “Conceptual Modeling – An Epistemological Foundation,” in *Proceedings of the Tenth Americas Conference on Information Systems (AMCIS 2004)*, N. C. Romano, Ed. Atlanta, Georgia, USA: Association for Information Systems, 06.08.2004 2004, pp. 4232–4242.
- [76] R. Dove, “Essays on Change Proficiency—Part Two: Business Engineering Principles and Knowledge Management,” *Paradigm Shift International*, Questa, NM, Essays 25–48, 1996–1998. [Online]. Available: <http://www.parshift.com/Files/Essays/EssayS02.zip>